Are LLM Written Unit Tests Flaky?

Progress Report 2, ECE 382V Software Testing in the Era of Nondeterminism, Fall 2026

Shaunak Kapur (sk62288)
The University of Texas at Austin
Austin, Texas, USA

Manas Pathak (map7848)
The University of Texas at Austin
Austin, Texas, USA

1 SUMMARY

The main problem after Progress Report 1 was that we had only 3 flaky LLM tests to study. We changed the plan to attack that shortage directly.

(1) **Known flaky code (E2).** We took projects where Gruber et al. [2] found a flaky test, reproduced that test on our own runner, and asked four LLMs to write tests for the same code without seeing any test file. E2 now covers 41 projects: 17 from their hand labeled sample and 24 from a replication batch that we chose by a fixed rule (Section 4.2).
(2) **More runs (E1).** We reran all 96 suites of the 12 library study, 600 times per generated test and 200 times per human test (Section 4.1).
(3) **A second model family.** We added two OpenAI models, GPT 5.4 mini and GPT 4.1 mini.
(4) **Why (E3).** For the flaky E2 tests we reran each one alone and searched for the test that pollutes it, as iDFlakies [4] does (Section 4.3).
(5) **Mitigation.** We tried two fixtures, three prompt versions, and longer rerun filters (Section 4.5).

We now have 74 flaky LLM tests that a developer would keep, plus 17 that a one run filter would remove (Table 1). Each has a root cause label. Three results stand out.

- **Each model fails in its own way** (Table 10). GPT 4.1 mini leaves randomness unseeded. GPT 5.4 mini is the only model that reloads modules, and 24 of its tests depend on test order because of it. In E2, Claude Haiku and Claude Sonnet flip only where the code under test is nondeterministic in a way a test cannot control.
- **Sonnet wrote the fewest flaky tests.** Over 41 projects, GPT 4.1 mini has a kept flaky test in 6 projects where Sonnet has none, and the reverse never happens (exact sign test, $p = 0.031$). Per suite it is 10 of 80 against 1 of 81 ($p = 0.005$). Only the suite level result survives a correction for six model pairs (Table 8).
- **A fixture fixes most of them.** Seeding every generator inside the test call removed all 12 unseeded randomness tests. Adding a state restore removed all 24 order dependent ones, and GPT 5.4 mini went from 25 kept flaky tests to 1. The 10 that are left, across all four models, depend on the clock or on a hash.

The 74 kept tests come from only 29 suites, so we compare models per suite and per project. Two of the 3 tests from Progress Report 1 flipped again and are inside the 15 of E1. E1 does not study removed tests, so that column is empty. In E2, 2 of the removed tests are not rejected themselves. They flip only after another test that the filter rejects (Section 4.3), so they would never flip in a filtered suite. The fixture runs of Section 4.5 rerun these same tests, so they add no new rows here.

Table 1: Flaky LLM tests we can now study. Kept means a one run generate and filter step would keep the test. The last three columns count the test functions, suites, and projects the kept tests come from.

Source	Kept	Removed	Func.	Suites	Proj.
E1, large rerun campaign	15	n/a	8	5	3
E2 plain, first 17 projects	13	12	13	7	5
E2 plain, replication (24)	36	2	30	12	6
E2 guard v1 and v3	10	3	8	5	3
Total	74	17	59	29	15

2 RESPONSE TO THE PROGRESS REPORT 1 FEEDBACK

Table 2 answers each comment.

3 STUDY DESIGN

Table 3 defines every condition. The *feature prompt* names the time, randomness, thread, and file features of each library so the model tests them. The *TestPilot style* prompt shows only signatures, docstrings, and README examples, following the TestPilot generator [9]. The *plain* prompt only asks for pytest unit tests for the given source. Every prompt is in `prompts/README.md`, word for word.

How suites were generated. E2 uses Claude Sonnet 5.5, Claude Haiku 5.5, GPT 5.4 mini, and GPT 4.1 mini. The Claude versions come from the Claude Code logs of each generation. Claude suites were written by Claude Code subagents, which read the source files themselves and were told not to open any test file. OpenAI suites came from the API, with the same source files pasted into the prompt, so those models never saw a test file. No generated test reuses the name of the project's flaky human test. The two pipelines differ in more than the model, which Section 4.6 discusses.

Flaky. A test is flaky if it both passed and failed on the same machine. Every run starts a fresh pytest process. Runs shuffle the test order (with the pytest-randomly plugin) and change the hash seed. Some runs also change the time zone or add CPU and memory load with stress-ng, using the configurations of Shaker [10], a tool that finds flaky tests by loading the machine. A generated test counts only if a one run generate and filter step would keep it, as in TestPilot, because that is what a developer would keep. A *test* is one pytest test case, so each parametrized case counts.

Table 2: Each comment on Progress Report 1 and what we changed

Comment	What we did
The subject filter numbers were packed into one sentence. What does "and we say so" mean?	The funnel is now a table (Table 4). The phrase meant that we oversample the first stratum on purpose. We now say that directly and give the reason below the table.
Why randomly sample, and why not run more?	The random sample was only for the last funnel rule, which needs a clone, an install, and a test run per package, so we checked 6 packages per stratum to estimate the pass rate. It did not pick our subjects. We now run far more (600 runs per generated test instead of 70 to 90), and instead of adding random libraries we added projects that are known to have flaky tests.
Why not use Python flaky tests that prior datasets already found?	We now do. We reproduced 21 of the 58 hand labeled flaky tests of Gruber et al. that are not order dependent and not caused by the network, then 30 of 176 more from their unlabeled data, and used 41 projects in all (Table 6).
Can a study rest on three flaky tests?	No, so we built E2 to find more. We now have 74 kept flaky LLM tests and 17 that a filter removes (Table 1). They cluster in a few suites, so we compare models per suite and per project.
Are LLMs just good at avoiding flakiness, and why?	Mostly, but not uniformly. Most LLM tests share no state between tests, which removes the main human cause (Section 4.4). Each model then has its own failure habit (Table 10), and we can now name each one. A fixture added after generation removes 39 of the 49 kept flaky E2 tests (Section 4.5).
Why only one shot Haiku?	Haiku 4.5 was only the first model where we saw flips. E2 compares four models from two families under the same plain prompt. There Claude Haiku 5.5 has 7 kept flaky tests in 1,908, all in two projects whose code is itself nondeterministic.
72 LLM suites does not equal 36 plus 24.	This was our error. There were 60 LLM suites (36 on October 1 and 24 on October 22) plus 12 human suites, so 72 suites in all. With the 24 OpenAI suites added since, E1 has 84 LLM suites and 96 suites in all.
What are AG-S, OS-S, OS-H?	Every condition is now defined in Table 3 before it is used.
The next steps did not address the shortage of flaky tests.	E2 was the first step, and its replication batch is now done (Table 6). Section 7 plans the rest.

Table 3: Conditions. All generation is one shot with no test execution, except AG-S.

Code	Model	Prompt
Human	library developers	none
AG-S	Claude Sonnet 5, agent that runs and fixes its tests	feature
OS-S	Claude Sonnet 5.5, one shot	feature
OS-H	Claude Haiku 4.5, one shot	feature
TP-S	Claude Sonnet 5.5, one shot	TestPilot style
PL-S	Claude Sonnet 5.5, one shot	plain
PL-G5	GPT 5.4 mini (OpenAI), one shot	plain
PL-G4	GPT 4.1 mini (OpenAI), one shot	plain

Three parts.

- **E1, 12 libraries.** Twelve Python libraries, each with its human suite and one suite per LLM condition (96 suites).
- **E2, matched study.** Projects where Gruber et al. found a flaky test. We reproduce that test, then ask each model to test the same code without showing it any test file.
- **E3, why.** Targeted reruns and a polluter search for every flaky generated test, a static comparison of how each condition builds its tests, and direct experiments.

We oversample stratum 1 on purpose, because nondeterminism enters through those modules. To be clear about the order of events, our 12 libraries were chosen during the pilot, before the funnel existed. We wrote the funnel after the proposal feedback so the choice can be checked and repeated. All 12 pass every rule. Ten are in stratum 1, and two, tinydb and tabulate, are in stratum 2 as controls.

Table 4: Subject funnel for E1 (top 5,000 PyPI packages)

Rule	Left
Top 5,000 by downloads (last 30 days)	5,000
Is a library (no stubs, pytest plugins)	4,701
Links a GitHub repository	4,131
Ships a pure Python wheel	3,296
At most 3 required dependencies, runs on Python 3.12	2,188
Repository has test files	1,959
Stratum 1, imports time, random, threading, asyncio, ...	1,135
Stratum 2, imports none of them	824
Suite passes offline in under 60 s (sample of 6 per stratum)	5 of 12

4 RESULTS SINCE PROGRESS REPORT 1

4.1 E1, the large rerun campaign

All E1 suites ran on GitHub Actions Linux runners with 2 virtual CPUs. Every run shuffles the order and the hash seed. Half the runs do only that, a quarter also set a random time zone, and a quarter add stress-ng. Generated tests ran 600 times and human tests, whose suites are about five times larger, 200 times, so a flip is easier to catch in an LLM test. Table 5 has the results. In all, 6.3 million test executions ran.

- **We cannot tell most LLM conditions apart from the human tests.** Without PL-G4, 5 of 7,468 LLM tests flipped (0.07%), against 5 of 5,587 human tests (0.09%). The intervals overlap almost completely, so any difference is small.
- **PL-G4 is the outlier, but from one cause.** Its 10 flaky tests are cases of three humanize test functions that depend on test order (Section 4.4). They are one bug repeated.

Table 5: E1 campaign. Generated tests are those the one run filter keeps. Human and AG-S rows count every test (AG-S already ran and fixed its own suite). Flaky % has a 95% Wilson interval.

Condition	Tests	Runs each	Flaky	Flaky % (95% CI)
Human	5,587	200	5	0.09 (0.04 to 0.21)
AG-S	1,511	600	1	0.07 (0.01 to 0.37)
OS-S	1,406	600	0	0.00 (0.00 to 0.27)
OS-H	1,595	600	3	0.19 (0.06 to 0.55)
TP-S	913	600	0	0.00 (0.00 to 0.42)
PL-S	1,191	600	0	0.00 (0.00 to 0.32)
PL-G5	852	600	1	0.12 (0.02 to 0.66)
PL-G4	900	600	10	1.11 (0.60 to 2.03)

- **The other LLM flaky tests are rare.** Two tenacity tests (AG-S and PL-G5) compare clock arithmetic with exact floating point equality and failed 6 and 2 times in 600 runs. Three OS-H tests depend on the time.

Progress Report 1 counted 7 human flaky tests, but two of them (filelock) flipped only on our macOS laptop, and E1 runs only on Linux. The five human flaky tests are two arrow locale tests that depend on test order, and three pybreaker tests that flipped only under stress-ng (two thread races and one timing test). PL-G4 also lost 2 of its 12 suites to imports of names the packages do not export.

4.2 E2, the matched study

Reproducing known flaky tests. Gruber et al. labeled by hand a sample of 100 flaky Python tests that are not order dependent. 42 are caused by the network, which leaves 58. Before any generation we fixed a rule for a second batch from their unlabeled data (`scripts/select_ext.py`), and we stopped when its candidate list ran out, not at a p value. From each of the 226 other projects with such a flaky test, we dropped the project if its name or every flaky test name points to the network, and otherwise took one test, preferring those not marked as infrastructure and with a failure rate closest to one half. For every test we checked out the project at the recorded commit, installed it in a fresh Python 3.8 environment, and ran the test up to 150 times (Table 6).

Table 6: Reproducing flaky tests of Gruber et al.

Outcome	Labeled	Replication
Tried (not order dependent, not network)	58	176
Repository gone or install failed	3	5
Always failed (old dependencies, data files)	21	107
Never failed (up to 150 runs)	13	34
Reproduced (passed and failed)	21	30
Used for the matched study	17	24

We left out 4 of the first 21 (two need downloads or a web API, and two Hypothesis tests mostly hit the time limit), and 6 of the 30 that still depend on a network service or on hardware. The replication tests had no label, so Manas labeled each human test after it reproduced. Of the 41 projects, 35 are randomness, and one each is time, concurrency, async wait, and floating point, with two in other categories. So E2 is still mostly about randomness.

Generating tests for the same code. For each project we found the code the flaky test exercises (`scripts/matched_targets.py`, then checked by hand) and asked each model for 10 to 25 tests of it, two independent samples per model. Every suite then ran 200 times on the same runner, half in file order and half shuffled. For the replication batch the runner also records which half each failure came from. The known flaky human test ran 200 times next to it as a control and failed at least once in 38 of 41 projects (860 failures in 8,200 runs). Two of the exceptions, docci and pyalcs, failed once in 150 runs during reproduction and not in these 200. The third, data-genie, failed in 98 of 150 runs during reproduction but in 0 of 500 later runs on fresh runners and once in 40 on a laptop. Its test has a 300 ms Hypothesis deadline, so we think its failures came from a slow runner session.

Table 7: E2 by model, 41 projects, plain prompt, 200 runs per test. Suites with one means suites with a kept flaky test. A few suites did not import, so the suite counts differ.

Model	Suites	Kept	Flaky kept	Suites with one	Removed
Claude Sonnet 5.5	81	1,883	1	1	1
Claude Haiku 5.5	81	1,908	7	4	0
GPT 5.4 mini	80	1,028	25	4	1
GPT 4.1 mini	80	1,070	16	10	12

Table 8: Model comparisons on the clustered units, per suite (Fisher's exact test on suites with a kept flaky test) and per project (exact sign test on projects where only one of the two models has one; the counts are first model : second model)

Pair	Suite p	Projects	Project p
Sonnet vs GPT 4.1 mini, first 17	0.053	0 : 4	0.13
Sonnet vs GPT 4.1 mini, replication 24	0.11	0 : 2	0.50
Sonnet vs GPT 4.1 mini, all 41	0.005	0 : 6	0.031
Haiku vs GPT 4.1 mini, all 41	0.10	1 : 6	0.13
Sonnet vs Haiku, all 41	0.37	0 : 1	1.0
Haiku vs GPT 5.4 mini, all 41	1.0	2 : 4	0.69
Sonnet vs GPT 5.4 mini, all 41	0.21	1 : 4	0.38
GPT 5.4 mini vs GPT 4.1 mini, all 41	0.16	3 : 6	0.51

"Removed" counts flaky tests a one run filter would drop, either because they failed in the first run or because they flip only after a test that did (2 GPT 4.1 mini tests, Section 4.3). Table 8 compares the models on suites and projects, because flaky tests cluster. The replication batch points the same way as the first batch for Sonnet against GPT 4.1 mini, and together they give $p = 0.005$ per suite. That survives a Bonferroni correction for the six model pairs (threshold 0.008), and the project level $p = 0.031$ does not. The suite level test treats a model's two suites for one project

as independent, so we read the project level result as the safer one. The other pairs are not significant. Per test numbers are in `results/matched_summary.json`, and we do not use them because GPT 5.4 mini's 25 tests come from just 4 suites.

In 28 of the 38 projects whose human test failed in our runs, no model wrote a kept flaky test. The human test was chosen because it is flaky, so this is not a rate comparison. E2 measures how often a model writes a flaky test when the code under test makes that easy.

4.3 Root causes

Manas labeled the root cause of every flaky LLM test with the categories of Luo et al. [6] and Gruber et al. Each label comes from the test, the code it calls, and a reproduction of the failure, and each was checked a second time against the code. Every label in `results/root_causes/` held up under that check, and the file keeps the quoted evidence and the reproduction for each. For every flaky E2 test but one (a removed gamble test) we also ran `scripts/od_diagnose.py`. It reruns the test alone 30 times, 10 of them with a fixed hash seed, and if the test never fails alone it runs each other test of the suite before it, as iDFlakies and iFixFlakies do. Table 9 has the counts.

Table 9: Root causes of the 91 flaky LLM tests. "Rem." means removed by a one run filter.

	E1	E2 plain		E2 guard	
Root cause	kept	kept	rem.	kept	rem.
Randomness, unseeded generator	0	12	9	4	2
Randomness, salted `hash()`	0	3	1	6	1
Unstable object hash in the code	0	6	1	0	0
Test order dependency	10	24	3	0	0
Time	3	4	0	0	0
Floating point on a clock value	2	0	0	0	0
Total	15	49	14	10	3

Table 10: Kept flaky E2 tests (plain prompt) by model and root cause

Root cause	Sonnet	Haiku	GPT 5.4	GPT 4.1
Unseeded generator	0	0	0	12
Salted `hash()`	0	3	0	0
Unstable object hash	1	4	0	1
Test order dependency	0	0	24	0
Time	0	0	1	3

Inherited from the code. In 24 of the 25 kept flaky E2 tests that are not order dependent, the test flips for the same reason as the project's human flaky test. Most randomness cases call code that draws from an unseeded global generator (NumPy, `random`, or PyTorch) and assert one outcome. In pelicun GPT 4.1 mini did seed NumPy at module level, but pytest-randomly reseeds NumPy before every test in the shuffled runs, so the seed had no effect there. The two hash causes are different. In cuckoopy the filter places items by Python's built in `hash()` of a string, which changes with `PYTHONHASHSEED` in every process, so seeding cannot help. In kevinarpe-rambutan3 the code's own `__hash__` returns the hash of a temporary `super` object, so it changes from call to call, and an `OrderedDict` that stores the object then raises `KeyError`. Every test that passes `None` to that check hits this bug, which is why all three models that test it flip there, as the human test does. The docci tests save the same workbook twice and compare the bytes, which differ when the two saves fall on either side of a two second zip timestamp boundary.

Introduced by the test. All 24 order dependent kept tests come from three GPT 5.4 mini suites, and none of them failed in a file order run. Each suite has a polluter that changes how modules are loaded and does not undo it. In torch-gpt-2, four tests monkeypatch a submodule attribute and then call `importlib.reload` on the package, which copies the fake into the package. The monkeypatch undoes only the submodule, so 7 later tests see the fake. In jmcm, two tests remove jmcm from `sys.modules` and import it again while `builtins.__import__` is patched to fail for NumPy or `math`. If one runs first, pandas, SciPy, and patsy are imported for the first time under the patch and stay half initialized in `sys.modules`. Then 13 later tests cannot import jmcm, and 3 other missing dependency tests get an `AttributeError` where they expect an `ImportError`. In pypfm, one test pops the package from `sys.modules` and imports a new copy, and a later test reloads the old copy. The polluter search agrees. Every one of the 24 passes alone in 30 runs, and for 23 of them a single earlier test makes it fail. The 24th, a jmcm test, fails after the NumPy polluter only when pytest-randomly has already imported NumPy, which our search runs without. The two GPT 4.1 mini victims in pylabeledrf follow an older pattern. A rejected polluter replaces a module level function with a fake and fails before restoring it.

E1 timing tests. The AG-S and PL-G5 tenacity tests build a fake timestamp from a real `time.monotonic()` reading and assert exact equality with the elapsed time, which fails for some clock values. One OS-H schedule test builds a clock time one hour ahead, which wraps past midnight when the run starts after 23:00 local time, so it failed only in the time zone mode.

4.4 E3, why most LLM tests do not flip

`scripts/characterize.py` scans every test function in E1 for shared state, isolation helpers, clock and sleep use, timing bounds, and randomness. Table 11 shows the features most tied to order dependence for the plain prompt conditions. All conditions are in `results/why_analysis.md`.

Why most LLM tests do not flip. Human suites share setup through class and module scoped fixtures (19.5% of human test functions), and 4.2% have setup code with no matching teardown. The order dependent human flaky tests sit on that kind of shared state, a class scoped fixture in the two arrow tests. No LLM condition uses a fixture wider than one test or has setup without teardown, and in every Claude condition 35 to 48% of tests build a fresh library object, against 18.5% of human tests.

Why GPT 4.1 mini flips in E1. Three of its humanize test functions (10 cases) call `monkeypatch.setattr(i18n._CURRENT, "locale", ...)`. `_CURRENT` is a `threading.local` whose `locale`

Table 11: Share (%) of E1 test functions with each feature (functions in the source, before the filter, so parametrized cases count once)

Feature	Human	PL-S	PL-G5	PL-G4
Test functions	3,077	927	897	985
Any shared state between tests	23.9	1.0	6.5	1.2
Class or module scoped fixture	19.5	0.0	0.0	0.0
Touches a private attribute	12.3	0.9	14.2	15.5
Resets library state after the test	0.3	8.6	9.8	0.0
Setup without teardown	4.2	0.0	0.0	0.0

attribute exists only after some earlier test has called `activate` or `deactivate`, so these tests pass only when such a test ran earlier. The one run filter ran the file in its written order, where they pass. PL-S resets this state around every humanize test, and PL-G4 never does. With one added fixture that calls `deactivate()` around every test, none of the 10 flipped in 50 shuffled runs, against all 10 without it (`scripts/humanize_reset_check.py`).

Table 12: E2 suites (82 per model, plain prompt) with each habit, from `scripts/matched_habits.py`

Habit	Sonnet	Haiku	GPT 5.4	GPT 4.1
Seeds a random generator	33	32	5	8
Uses `monkeypatch` or `mock.patch`	21	27	44	30
Reloads a module or edits `sys.modules`	0	0	13	0

Why GPT 4.1 mini flips in E2. The code under test draws random numbers itself, and Table 12 shows that the GPT models rarely seed. GPT 5.4 mini still wrote no unseeded randomness test, because it replaces the random draws with fakes through `monkeypatch`. Sonnet and Haiku seed in about 40% of suites and often pick inputs that avoid the random path, as with `squeeze=True` in spreading_dye_sampler. GPT 4.1 mini does neither in most suites.

Why GPT 5.4 mini flips in E2. Its suites test the package itself, not only its functions. 13 of its 82 suites reload a module or edit `sys.modules`, and no suite of the other three models does. 4 of those 13 have a flaky test (the fourth, limix-lmm, is rejected by the filter). None of them restores the module state afterwards, and the file order filter run hides the problem.

Why Haiku 4.5 flipped in Progress Report 1. With the same prompt as one shot Sonnet, it wrote about twice as many test functions, never froze the clock in schedule where Sonnet usually did, never injected a fake sleep in tenacity, and wrote one sided timing bounds with less headroom (counts in `results/why_analysis.md`).

4.5 Mitigation

We tried three kinds of mitigation, research question 4 of the proposal, on the E2 tests.

Fixtures added after generation. A fixture is a `conftest.py` added to each generated suite, with the tests unchanged. *M1* (`mitigations/seed_conftest.py`) seeds `random`, NumPy, and PyTorch inside each test call. It has to run there, after pytest-randomly reseeds, which is also why the module level seeding GPT 4.1 mini wrote in pelicun had no effect. *M2* (`mitigations/isolate_conftest.py`) does the same, imports the project's packages once before any test runs, and snapshots `sys.modules`, the globals of every loaded module, `sys.path`, the environment, and the working directory before each test, then restores whatever changed after teardown. The up front import is there for jmcm. Without it, a restore cannot help, because the polluter is the first to import pandas and leaves it half built (in a local check of 40 shuffled runs, a restore alone did not help, and adding the up front import removed every failure). Every suite reran 200 times as before (Table 13).

Table 13: Kept flaky E2 tests with each fixture, 41 projects, kept tests in parentheses

Model	Plain	M1, seed	M2, seed and restore
Claude Sonnet 5.5	1 (1,883)	2 (1,884)	2 (1,884)
Claude Haiku 5.5	7 (1,908)	5 (1,906)	5 (1,906)
GPT 5.4 mini	25 (1,028)	25 (1,028)	1 (1,027)
GPT 4.1 mini	16 (1,070)	3 (1,075)	2 (1,073)

M1 removed all 12 unseeded generator tests. Kept counts moved by at most 5 per model. GPT 4.1 mini keeps 5 more because some tests that failed their first run now pass it, and Haiku's 2 fewer are kevinarpe-rambutan3 tests that happened to fail their first run. M2 removed all 24 order dependent tests (jmcm, torch-gpt-2, and pypfm) and kept every M1 fix, so GPT 5.4 mini goes from 25 to 1. It made no new flaky test, since every test still flaky under M2 was also flaky in the plain or M1 run, and kept counts moved by at most 3, so the restore did not break any suite. The 10 kept flaky tests left under M2 are all in three projects. What neither fixture can fix is the clock (docci), the salted `hash()` (cuckoopy), and the code's own unstable hash (kevinarpe-rambutan3, whose counts move between runs because its failures depend on memory layout, which is also why Sonnet goes from 1 to 2 under both fixtures). These need a change to the test or to the code.

Prompts. We regenerated the 34 GPT 4.1 mini suites of the first 17 projects with three versions of a guard prompt (`prompts/README.md`). Version 1 lists the usual causes of flakiness. Version 2 adds three concrete rules (seed inside the test body, do not assert on `hash()` of strings, use `monkeypatch` for module attributes). Version 3 limits seeding to the generators the code under test already uses, because version 2 made GPT 4.1 mini import PyTorch or NumPy into projects that do not have them.

The concrete rules work, but they cost suites. Guard v3 cut kept flaky tests from 10 to 1, but 5 of 34 suites did not import and the filter rejected a third of the tests, so 129 fewer tests survive than with the plain prompt. The one remaining test is a cuckoopy `hash()` case. The guard prompts also led GPT 4.1 mini to write more cuckoopy tests that hit `hash()` (6 kept in v1 and v3, none with the plain prompt), because they ask for tests of collisions and deletes. M1 removed the same 10 with no loss, so for unseeded randomness a fixture is the better tool.

Table 14: GPT 4.1 mini guard prompts, first 17 projects (34 suites)

Prompt	Did not import	Kept	Flaky kept
Plain	0	558	10
Guard v1	0	550	9
Guard v2	9	500	0
Guard v3	5	429	1

Longer rerun filters. From the measured failure rate of each of the 74 kept flaky LLM tests, we estimated what k extra filter runs would catch, separately for runs in file order and in shuffled order (`scripts/krun_estimate.py`, Table 15). File order rates are measured for the 37 tests from the replication batch and guard v3. For the others we use the pooled rate, except the 10 PL-G4 humanize tests, which always pass in file order (Section 4.4). With that, 40 of the 74 never fail in file order, so a filter that only repeats the file order catches about a quarter of them even with 10 runs. Shuffling the extra runs nearly triples that. Stable tests cost only run time, because they never fail.

Table 15: Expected share (%) of the 74 kept flaky LLM tests that k extra filter runs would catch

Extra runs k	1	3	5	10
File order	7	16	20	26
Shuffled order	22	47	60	74

4.6 Threats to validity

- **Pipeline, not only model.** Claude suites came from Claude Code subagents that read files, and OpenAI suites came from the API with sources pasted in. Sampling settings and system prompts differ too, so the model comparison is a comparison of two pipelines with four models.
- **Scope of E2.** 35 of 41 projects are randomness projects. Each of time, concurrency, async wait, and floating point has one project. No model wrote a flaky test in the last three, and the four time tests are all in docci, so these say little yet.
- **Clustering and multiple tests.** Flaky tests cluster, so we compare suites and projects, not tests. We report six model pairs, and only the suite level Sonnet against GPT 4.1 mini result survives a correction for that.
- **Unequal run budgets in E1.** Generated tests ran 600 times and human tests 200 times, which favors finding LLM flaky tests. E2 gives both 200 runs.
- **What M2 restores.** M2 undoes changes to module state that a test may make on purpose. It removed flips in our suites, but it could also hide a real bug that leaks state, so we see it as a filter aid, not a fix to ship.
- **Old environments and few samples.** E2 runs on Python 3.8 with old pinned dependencies, and each model wrote only two suites per project.
- **Generation and labeling checks.** We told the Claude subagents not to open test files and checked that no generated test reuses the flaky test's name, but we did not audit every file they read. Whoever labeled a test could see which model wrote it, and each label had one labeler and one second check, so we have no agreement statistic yet.

5 MILESTONE GOALS AND STATUS

Table 16 lists the Intermediate Steps 2 goals from the proposal and the next steps we promised in Progress Report 1.

6 CHALLENGES

- **Old projects are hard to run.** Only 51 of 234 known flaky tests reproduced. Most of the rest always failed because of old dependencies or missing data files.
- **Harness bugs cost us two projects for a while.** fuzzylogic's own tests add its `src` folder to the import path by hand, and fossor's settings tell pytest to collect only files under `test/`. Our runner handled neither, so every generated suite in those projects collected nothing. We fixed both and reran them.
- **Models invent names.** PL-G4 lost two whole E1 suites to imports of names the packages do not export, and in pymemorydb three models imported the package by its project name, which is not its import name.
- **A one run filter in file order hides order dependence.** That is how all 34 order dependent kept tests got through.

7 NEXT STEPS AND RECOVERY PLAN

The steps are in priority order. Progress Report 3 is due November 20.

(1) **Mitigation for every model and E1 (Shaunak, by November 13).** Run M1 and M2 on the E1 suites, run guard v3 for the other three models on E2, and measure a 3 and 5 run shuffled filter directly instead of estimating it. For each we report flaky tests kept, good tests lost, coverage, the failure rate of the flaky tests left, the scanner's risk rate, and cost. To separate model from pipeline, we will also regenerate the Sonnet and Haiku E2 suites with `claude -p`, no tools, and the same pasted source prompt as the OpenAI models.
(2) **An order dependent arm (Manas, by November 20).** Gruber et al. also found 4,461 order dependent tests in 615 projects. For 10 of these projects (5 if time runs short) we will generate tests for the same code and run them inside the human suite in shuffled orders. GPT 5.4 mini's reload habit makes this sharper, because its tests could become polluters of human tests.
(3) **Non randomness projects (Shaunak, by November 20).** E2 has only six projects whose flaky test is not about randomness. We will reproduce more time and concurrency tests from the unlabeled data, starting with test files that import time, threading, or asyncio.
(4) **Root causes (Manas, ongoing).** Label every new flaky test the same way as above. Shaunak will label a random sample of 20 without seeing the model, so we can report agreement.

Table 16: Goals for this milestone

Goal	Status	Notes
Rerun every flagged test at least 1,000 times on the GitHub runners, with and without CPU stress	Done	Done for Progress Report 1 (over 1,300 runs each). Every E1 test now also has 600 (generated) or 200 (human) runs.
Add a non Claude model	Done	GPT 5.4 mini and GPT 4.1 mini with the plain prompt, in E1 and E2. We did not run them with the TestPilot style prompt, and spent that budget on E2 instead.
Add the plain prompt	Done	Added October 22 (PL-S), now also used for every E2 model.
Measure scanner precision and recall	Done	Done for Progress Report 1 on the hand labeled tests of Gruber et al. Recall is low (10 of 96), because most of the nondeterminism is inside the library, not in the test. It flagged 35 of 714 tests that were never reported flaky (4.9%).
Measure coverage of every suite	Done	Mean line coverage is 94.4% for human suites, 82.6% for PL-S, 75.6% for PL-G5, and 67.0% for PL-G4, two of whose suites did not import. We leave out filelock, whose Windows module never runs on our machines (`results/coverage/`).
Search for order dependent tests in the style of iDFlakies [4], a tool that reruns tests in many orders to find order dependence	Done for E2	Every rerun shuffles the order, and `scripts/od_diagnose.py` finds the polluters of the order dependent E2 tests. Progress Report 1's mixed suites (generated tests run inside the human suite) found three AG-S schedule victims of human state. The order dependent arm of Section 7 extends that to E2.
Add four new subjects from the funnel	Replaced	E2 adds 41 projects with known flaky code, which attacks the shortage of flaky tests more directly.
Pick stress-ng configurations with Shaker's selection step	Not done	Only three human tests flip only under stress-ng, too few to choose configurations from.
Run FlaPy [1] (the rerun tool of Gruber et al.) on every suite	Dropped	FlaPy ran on two suites. Our harness already reruns in fresh processes in file and shuffled order, which is what FlaPy does.
Start the mitigation early	Done	Two fixtures, three prompt versions, and the rerun filter estimate (Section 4.5).

(5) **Delay injection (if time allows).** Use FlakeRake [8], which inserts delays to reproduce timing failures, on the E1 tests with timing bounds.

Check point and fallback. On November 13 we check steps 1 and 2. Our reproduction yield so far is 51 of 234 (22%), so step 2 will try about 45 order dependent tests to get 10 projects. If fewer than 5 have reproduced by then, we stop at the ones that did and move that time to step 3. Step 5 is the first to cut.

Division of work. Since Progress Report 1, Manas did the literature review and the root cause labeling of the new flaky tests, including the human tests of the replication batch. Shaunak built the reproduction and matched study pipeline, generated the OpenAI suites, ran the E1 campaign and the mitigation trials, and wrote the analysis scripts and this report. Manas will also write the related work section of the final report.

8 RELATED WORK

Luo et al. [6] give the root cause categories we use, and Gruber et al. [2] show what flakiness looks like in Python and provide the tests behind E2. TestPilot [9] is the generate and filter pipeline we imitate, and CodaMosa [5] uses an LLM to help a search based test generator. Both keep a test after one passing run, which Section 4.5 shows is not enough for flaky tests. Shaker [10] gives our stress-ng configurations, and iDFlakies [4] the shuffled reruns and the polluter search. FlakeRake [8] is the delay injection we plan. Parry et al. [7] survey the field. The closest study is Gruber et al. [3], who measured flakiness in tests from search based generators such as EvoSuite and Pynguin. We ask the same question for LLM generators, compare against the human tests of the same code, and test mitigations in the generate and filter step.

9 CODE, DATA, AND DOCUMENTATION

Everything is in the public repository https://github.com/shaunak1206/ece382v-flaky-llm-tests.

- `README.md` explains how to set up the environment and gives the command for every experiment.
- `docs/DATA.md` documents every result file and field. Raw results are under `results/`, generated suites under `gen_*`, `llm_tests/`, and `matched/`, prompts in `prompts/README.md`, and root cause labels in `results/root_causes/`.

New since Progress Report 1:

- `scripts/gruber_reproduce.py` and the `gruber-reproduce` workflow reproduce a known flaky test at its original commit. `scripts/select_ext.py` picks the replication batch.
- `scripts/matched_targets.py` finds the code a flaky test exercises. `scripts/matched_run.py` and the `matched-study` workflow rerun generated suites next to the human flaky test. `scripts/analyze_matched.py` and `scripts/matched_stats.py` make the E2 tables.
- `scripts/od_diagnose.py` and the `od-diagnose` workflow run the alone and polluter reruns.
- `mitigations/` holds the M1 and M2 fixtures, `scripts/make_seed_suites.py` adds them to every suite, and `scripts/krun_estimate.py` makes the filter estimate.
- `scripts/gen_openai.py` generates the OpenAI suites, including every guard prompt, and logs every call.
- `scripts/scale_reruns.sh`, the `scale-reruns` workflow, and `scripts/analyze_scale.py` run

the E1 campaign. `scripts/characterize.py` and `scripts/humanize_reset_check.py` support E3.

REFERENCES

[1] Martin Gruber and Gordon Fraser. 2023. FlaPy: Mining Flaky Python Tests at Scale. In *IEEE/ACM International Conference on Software Engineering (ICSE), Companion Proceedings, Tool Demonstrations*.

[2] Martin Gruber, Stephan Lukasczyk, Florian Kroiß, and Gordon Fraser. 2021. An Empirical Study of Flaky Tests in Python. In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*. 148–158.

[3] Martin Gruber, Muhammad Firhard Roslan, Owain Parry, Fabian Scharnböck, Phil McMinn, and Gordon Fraser. 2024. Do Automatic Test Generation Tools Generate Flaky Tests?. In *IEEE/ACM International Conference on Software Engineering (ICSE)*.

[4] Wing Lam, Reed Oei, August Shi, Darko Marinov, and Tao Xie. 2019. iDFlakies: A Framework for Detecting and Partially Classifying Flaky Tests. In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*. 312–322.

[5] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In *IEEE/ACM International Conference on Software Engineering (ICSE)*.

[6] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An Empirical Analysis of Flaky Tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. 643–653.

[7] Owain Parry, Gregory M. Kapfhammer, Michael Hilton, and Phil McMinn. 2021. A Survey of Flaky Tests. *ACM Transactions on Software Engineering and Methodology* 31, 1 (2021).

[8] Shanto Rahman, Aaron Massey, Wing Lam, August Shi, and Jonathan Bell. 2024. Automatically Reproducing Timing-Dependent Flaky-Test Failures. In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*.

[9] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *IEEE Transactions on Software Engineering* 50, 1 (2024), 85–105.

[10] Denini Silva, Leopoldo Teixeira, and Marcelo d'Amorim. 2020. Shake It! Detecting Flaky Tests Caused by Concurrency with Shaker. In *IEEE International Conference on Software Maintenance and Evolution (ICSME)*.